

Lectura 2

Introducción a MATLAB/Octave/Scilab

Primera Parte. Operaciones.

Dr. Luciano Ponzellini Marinelli
lucianoponzellini@uca.edu.ar

Métodos y Cómputos Numéricos
Licenciatura en Ciencias de Datos
Facultad de Química e Ingeniería
Rosario, Argentina

Notas de clase
5 agosto 2025





Organización

CARACTERÍSTICAS

OPERACIONES

FUNCIONES PREDEFINIDAS

PROGRAMACIÓN

CARACTERÍSTICAS

Características principales

- Software matemático (gran calculadora) para realizar cálculos numéricos con vectores y matrices.
- Trabaja tanto en $\mathbb{R}$ como en $\mathbb{C}$ para representar datos.
- Tiene un entorno de desarrollo integrado para facilitar el manejo del software y un lenguaje de programación propio al implementar algoritmos.
- Permite realizar gran variedad de gráficos 2D y 3D para representar funciones.
- Puede usarse en plataformas de software libre (GNU/Linux) y plataformas de software privados (Windows, Mac, etc).
- Permite crear interfaces de usuario (GUI), paralelizar cálculos, comunicarse con programas en otros lenguajes y dispositivos.

Entorno de MATLAB/Octave/Scilab

- Al iniciar (doble clic en el acceso directo) aparece la **Ventana de Comandos (Command Window)** donde se ejecutan los comandos.
- El **Editor de textos (Editor)** propio permite escribir programas.
- En el **Area de Trabajo (Workspace)** se visualiza las variables utilizadas en nuestra Ventana de Comandos.
- El **Centro de Ayudas (Help)** puede visualizarse opcionalmente o bien acceder desde la Ventana de Comandos.
- El **Historial de Comandos (Command History)** permite tener un historial de comandos utilizados.
- El **Perfilador (Profiler)** permite optimizar y depurar la performance de un algoritmo.
- El **Depurador (Debugger)** permite eliminar errores de programación.

¿Cómo es el entorno MATLAB?

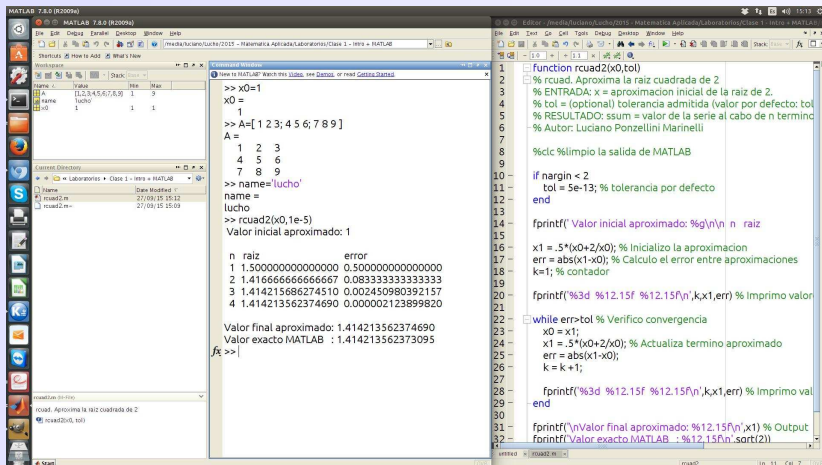


Figura: Entorno de trabajo en MATLAB.

¿Cómo es el entorno Octave?

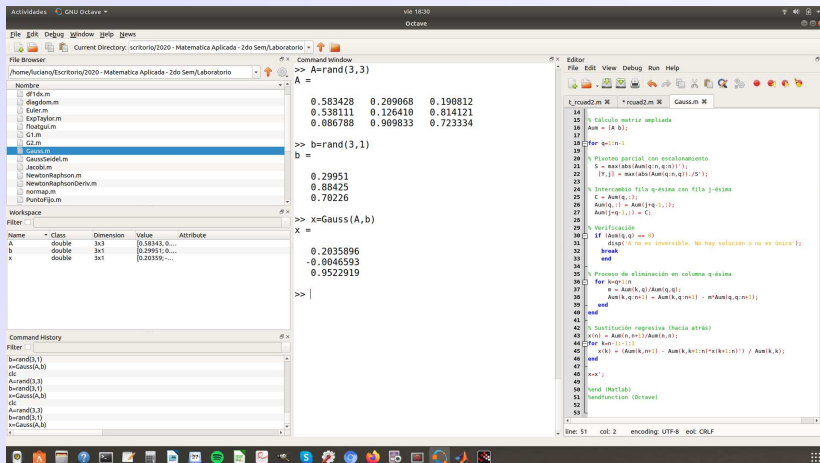


Figura: Entorno de trabajo en Octave.

¿Cómo es el entorno Scilab?

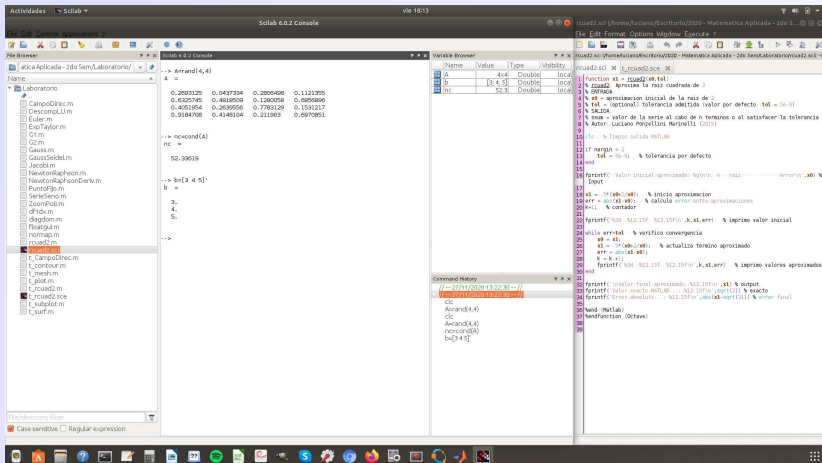


Figura: Entorno de trabajo en Scilab.

¿Cómo trabajar las expresiones?

FORMA 1:

Con el operador asignación (=) asignamos valor(es) a variable(s)

variable = expresión

```
>> phi = (1+sqrt(5))/2;
```

El punto y coma (;) evita que el resultado se imprima por pantalla.

FORMA 2:

expresión

```
>> (1+sqrt(5))/2;
```

Acá almacenamos el resultado a una variable interna llamada **ans** (**answer**) hasta que dicha variable se actualice.

¿Cómo trabajar en Ventana de Comandos?

- Una expresión termina con **Enter**.
- Si deseamos continuar escribiendo en la próxima línea, introducimos los **tres puntos (...)**.
- Podemos incluir varias expresiones en la misma línea separando por **coma (,)** o **punto y coma (;)**.
- Para limpiar la Ventana de Comandos uso el comando **clc**.
- Eliminamos todas las variables de la memoria usando **clear**.
- Si solo quiero borrar alguna variable determinada, hacemos:

```
>> clear nom_var
```

¿Cómo muestro mi salida numérica?

- Los números reales se calculan en doble precisión almacenados en 8 bytes: 53 bits para la mantisa, 11 para el exponente en base 2 y uno para el signo (entre 15 y 16 cifras decimales).
- MATLAB/Scilab/Octave calculan internamente con máxima precisión.
- Se pueden modificar los formatos numéricos mostrados en pantalla.

format short Coma fija escalada con 4 decimales (por defecto).

MATLAB, Octave:

```
>> pi
ans = 3.1416
```

Scilab:

```
- -> %pi
%pi = 3.1415927
```

format long Coma fija escalada con 15 decimales (Matlab) o 14 (Octave)

MATLAB, Octave:

```
>> format long, pi
ans = 3.141592653589793
```

Scilab:

```
- -> format(25), %pi
%pi = 3.141592653589793115998
```

Más ejemplos de formatos numéricos en MATLAB

format short e Notación científica de punto flotante con 4 decimales.

```
>> format short e, pi  
ans = 3.1416e+00
```

format long e Idem con 14 decimales.

```
>> format long e, pi  
ans = 3.141592653589793e+00
```

format short eng Formato ingenieril con 4 dec. y potencia múltiplo de 3.

```
>> format short eng, 100*pi  
ans = 314.1593e+000  
>> 1000*pi  
ans = 3.1416e+003
```

format long eng Formato ingenieril con 14 dec. y potencia múltiplo de 3.

```
>> format long eng, pi  
ans = 3.14159265358979e+000
```

Tipos de datos: *eps*, *realmin*, *realmax*

eps: devuelve la diferencia entre 1 y el número inmediato superior.

MATLAB, Octave:

```
>> eps  
ans = 2.2204e-016
```

Scilab:

```
- -> %eps  
%eps = 2.220D-16
```

realmin: devuelve el menor número positivo con que se puede trabajar.

MATLAB, Octave:

```
>> realmin  
ans = 2.2251e-308
```

Scilab:

```
- -> number_properties ("tiny")  
ans = 2.23D-308
```

realmax: devuelve el mayor número positivo con que se puede trabajar.

MATLAB, Octave:

```
>> realmax  
ans = 1.7977e+308
```

Scilab:

```
- -> number_properties ("huge")  
ans = 1.80D+308
```

Tipos de datos: *Infinito (Inf)* y *NaN (Not a Number)*

- Hay un formato especial para números muy grandes (que no puede representar) considerados infinito:

```
>> 1.0/0.0
```

Warning: Divide by zero (algunas versiones)

```
ans = Inf
```

- También hay una representación especial para resultados que no están definidos como números:

```
>> 0/0
```

Warning: Divide by zero (algunas versiones)

```
ans = NaN
```

```
>> Inf/Inf
```

```
ans = NaN
```

¡Hay un álgebra de *Inf* y *NaN*!

¿Cómo trabajar con números complejos?

- MATLAB/Octave/Scilab trabajan sin ninguna dificultad en $\mathbb{C}$:

```
>> sqrt(-1)
ans = 0 + 1.0000i

>> 2-3i
ans = 2.0000 - 3.0000i

>> 1+2*j
ans = 1.0000 + 2.0000i  (j no debe ser estar predeterminada)
```

- Con **complex** creamos un complejo en binario a partir de sus dos argumentos (*parte real e imaginaria*):

```
>> z = complex(3,2)
z = 3.0000 + 2.0000i
```

¿Cómo leer por pantalla?

- Podemos escribir mensajes por pantalla para definir variables desde el teclado y luego sean almacenadas en nuestra area de trabajo.
- La función **input** imprime un mensaje en la línea de comandos y recupera como valor de retorno un valor numérico (escalar, vector, matriz) o un string (entre apóstrofes).
- Después de imprimir el mensaje el programa espera que el usuario ingrese un valor:

```
>> n = input('Cantidad de ecuaciones del sistema = ') (n numérico)
Cantidad de ecuaciones del sistema = 10
>> n = 10

>> s = input('Ingrese nombre del método (entre apóstrofes): ') (s
string)
Ingrese su nombre del método (entre apóstrofes): 'Gauss'
>> s = 'Gauss'
```

¿Cómo imprimir por pantalla?

- La función **disp** imprime en pantalla un mensaje de texto o el valor de una variable pero sin imprimir su nombre:

```
>> disp('Fin del programa')
```

```
Fin del programa
```

```
>> A=rand(2,5);
```

(defino una matriz aleatoria 2×5)

```
>> A
```

```
A =
```

```
0.9340  0.7577  0.3922  0.8235  0.3171
```

```
0.6787  0.7431  0.6555  0.6948  0.9502
```

```
>> disp(A)
```

(muestro solo los coeficientes)

```
0.9340  0.7577  0.3922  0.8235  0.3171
```

```
0.6787  0.7431  0.6555  0.6948  0.9502
```

¿Cómo escribir por pantalla?

- La función **fprintf** imprime en pantalla mensajes y valores en distintos formatos. Su estructura es:

```
fprintf('texto')
fprintf('cadena de control', var1, var2,...)
```

```
>> s = 'Matemática Aplicada', fprintf(s) (en Scilab es printf)
```

```
Matemática Aplicada>>
```

```
>> fprintf('Matemática Aplicada')
```

```
Matemática Aplicada>> (En MATLAB/Octave el prompt queda pegado)
```

```
>> fprintf('Matemática Aplicada\n')
```

```
Matemática Aplicada (el \n baja el prompt)
```

```
>>
```

¿Cómo escribir por pantalla?

- La 'cadena de control' contiene los formatos de escritura (ver help):
 - `%d`, `%i`, `%u`, `%o`, `%x`, `%X` para números enteros.
 - `%f`, `%e`, `%E`, `%g`, `%G` para números punto flotante.
 - `%c`, `%s` para cadenas de caracteres.
- Se puede controlar la salida numérica con estas opciones:

```
>> raiz = 1.414213562374690;  
>> fprintf('Valor final aproximado: %12.5f \n', raiz)  
Valor final aproximado: 1.41421  
(12 dígitos antes de la coma y 5 después en formato de coma fijada)  
  
>> fprintf('Valor final aproximado: %12.5e \n', raiz)  
Valor final aproximado: 1.41421e+00  
(12 dígitos antes de la coma y 5 después en formato exponencial)
```

OPERACIONES

Operadores aritméticos

- MATLAB/Octave/Scilab operan variables por medio de **operadores**.
- Los **operadores aritméticos** son los siguientes:

+	adición o suma
-	sustracción o resta
*	multiplicación
/	división
^	potenciación
\	división inversa

Ejemplo:

```
>> c= 3 \ 1  
c = 0.3333
```

(divide de derecha a izquierda)

Operadores lógicos

- Disponen de los siguientes **operadores lógicos**:

& **y (and)**
| **o (or) no excluyente**
xor **o excluyente**
~ **negación lógica**

Ejemplos:

```
>> 1&1  
ans = 1            (verdadero)  
  
>> 1&0  
ans = 0            (falso)  
  
>> xor(1,1)  
ans = 0
```

Operadores relacionales

- Disponen de los siguientes **operadores relacionales**:

< menor que
> mayor que
<= menor o igual que
>= mayor o igual que
== igual que
~= distinto que

Ejemplos:

```
>> 1==1  
ans = 1            (verdadero)  
>> 3.1415926==pi  
ans = 0            (falso)  
>> 3.1415926~=pi  
ans = 1
```

¿Cómo trabajar con *vectores* y *matrices*?

$$\mathbf{x} = [x_1, \dots, x_n] \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \quad \mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

- Matrices y vectores se definen como **variables con nombres**.
- MATLAB/Octave/Scilab distinguen mayúsculas y minúsculas.
- No hace falta predefinir el tipo de variable.
- No hace falta preestablecer su dimensión (se puede modificar).
- Según se la defina se determina su dimensión.

Definición de vectores

- Podemos definir un **vector fila** **x** separando sus componentes por espacios en blancos o bien por comas (,):

```
>> x = [1 2 4]    (o bien x = [1, 2, 4])  
x =  
1    2    4      (vector fila)
```

- Si los números están separados por punto y coma (;) obtenemos un **vector columna** **y**:

```
>> y = [1; 3; 9]  
y =  
1  
3      (vector columna)  
9
```

Definición de matrices

- Definimos las **matrices** por filas separando los elementos de una misma fila por espacios en blancos o comas (,) y separando las filas por punto y coma (;):

```
>> A=[8 1 6; 3 5 7; 4 9 2]
```

```
A =
```

```
8   1   6
```

```
3   5   7
```

```
4   9   2
```

- Internamente las matrices se almacenan por columnas en una columna (aunque se introduzcan por filas).
- Existen una gran variedad funciones predefinidas que generan de matrices (rand, magic, hilb, ...).

Definición de matrices

- Se puede crear una nueva matriz o vector a partir de otras matrices o vectores definidos previamente:

```
>> z = [x 8 16 32]
z = 1   2   4   8  16  32
>> B = [A; x]

B =
8   1   6
3   5   7
4   9   2
1   2   4
>> C = [B; z]
```

Error using ==_z vertcat

CAT arguments dimensions are not consistent.

¿Cómo acceder a los elementos de un vector o

- Accedemos a los elementos de un vector con el índice entre paréntesis:

```
>> z(5)  
ans = 16  
>> z(7)
```

Attempted to access z(7); index out of bounds because numel(z)=6.

- Accedemos a los elementos de una matriz con los índices entre paréntesis separados por coma:

```
>> B(2,3)  
ans = 7
```

- También puede accederse con un sólo subíndice:

```
>> B(6)  
ans = 5 (¿por qué?)
```

Otra forma de crear vectores y matrices

- Podemos crear vectores o matrices dando valores de algún/os de sus elementos. Automáticamente se asigna cero a todos los demás elementos del vector o matriz.

```
>> u(3) = .1, u(10) = .5
u = 0 0 .1 0 0 0 0 0 0 .5
```

```
>> C(3,5) = 1
C =
0 0 0 0 0
0 0 0 0 0
0 0 0 0 1
```

- Podemos definir la componente c_{23} de la matriz **C** haciendo:

```
>> C(2,3) = 1
```

Operaciones con vectores y matrices: *trasposición*

- El operador apóstrofo (') es la **trasposición matricial**:

```
>> B'
ans =
 8   3   4   1
 1   5   9   2
 6   7   2   4
```

- MATLAB diferencia entre vectores fila y vectores columna:

```
>> x+y
Error using ==: +
Matrix dimensions must agree.
Lo correcto sería:
>> x+y' (vector fila)
>> x'+y (vector columna)
```

Operaciones con vectores y matrices: *producto interno*

- Podemos definir el **producto escalar interno** entre vectores del mismo espacio vectorial:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x} \cdot \mathbf{y}^T = \sum_{i=1}^n x_i y_i \quad \mathbf{x}, \mathbf{y} \in \mathbb{R}^n.$$

- Podemos definirlo usando el producto adecuadamente:

```
>> x*y
ans = 43
>> x*y'
Error using ==: mtimes
Inner matrix dimensions must agree.
>> x'*y
Error using ==: mtimes
Inner matrix dimensions must agree.
```

Operaciones con vectores y matrices: *producto externo*

- Si en cambio trasponemos los vectores **x** e **y**:

```
>> xt=x';    (x pasa a vector columna)
>> yt=y';    (y pasa a vector fila)
>> xt*yt
ans =
1   3   9
2   6  18
4  12  36
```

Se denomina **producto escalar externo** (genera una matriz).

- Podemos combinar operadores:

```
>> x*2
ans = 2   4   8    (multiplico por 2 cada componente)
>> x-1
ans = 0   1   3    (resto 1 a cada componente)
```

Operaciones con vectores y matrices: *el operador punto (.)*

- Para multiplicar o dividir vectores o matrices *componente a componente*, utilizamos el **operador punto** **(.*)** y **(./)**:

```
>> x.*x  
ans = 1   4   16  
>> c=[1 0 1]  
c = 1   0   1  
>> x./c  
ans = 1   Inf   4
```

- Análogamente para la potencia **(.^)**:

```
>> x.^ 3  
ans = 1   8   64
```

Operaciones con vectores y matrices: *el operador punto (.)*

- Análogamente, consideremos las matrices:

```
>> M = [1 2 3; 4 5 6]
```

```
M =
```

```
1 2 3
```

```
4 5 6
```

```
>> N = [1 2; 3 4; 5 6]
```

```
N =
```

```
1 2
```

```
3 4
```

```
5 6
```

- Para obtener el producto matricial **M.N** hacemos:

```
>> M*N
```

```
ans =
```

```
22 28
```

```
49 64
```

Operaciones con vectores y matrices: *el operador punto (.)*

- En cambio, si hacemos:

```
>> M*N'
Error using ==: mtimes
Inner matrix dimensions must agree.
```

- Con el operador $(.)$ obtenemos el producto por componentes:

```
>> M.*N'
ans =
1   6  15
8  20  36
```

- Análogamente para la potencia $(.^)$:

```
>> M.^ .5
1.0000   1.4142   1.7321
2.0000   2.2361   2.4495
```

Operaciones con vectores y matrices: *el operador dos puntos (:)*

- Utilizamos el **operador dos puntos (:)** para representar rangos:

[inicio:paso:fin]

```
>> i = 1:10
i = 1  2  3  4  5  6  7  8  9 10
>> j = 1:2:10
j = 1  3  5  7  9
>> k = 1:1.5:10
k = 1.0000  2.5000  4.0000  5.5000  7.0000  8.5000  10.0000
>> h = 10:-1:1
h = 10  9  8  7  6  5  4  3  2  1
```

Este operador resulta muy útil para definir “discretizaciones”.

Operaciones con vectores y matrices: *el operador dos puntos (:)*

- Este operador es muy potente para matrices:

```
>> A(3, :)
```

```
ans = 4 9 2
```

(devuelve la 3er. fila de **A**)

```
>> A(:,2)
```

```
ans =
```

```
1
```

```
5
```

```
9
```

(devuelve la 2da. columna de **A**)

```
>> A(2:3, 2:3)
```

```
5 7
```

```
9 2
```

(devuelve una submatriz 2x2 de **A** desde (2,2) a (3,3))

Operaciones con vectores y matrices: *el operador dos puntos (:)*

- Podemos usar este operador (:) para invertir el orden de elementos de un vector o permutar filas/columnas:

```
>> x = x(3:-1:1)
x =  4  2  1
>> D = A(:,3:-1:1)
D =
 6  1  8
 7  5  3
 2  9  4      (permutamos las columnas de A)
```

- Si usamos corchetes [] dentro del paréntesis, hacemos referencia a los *índices* del vector o matriz:

```
>> A([1 3])
ans = 8  4      (se conoce como "indexación")
```

FUNCIONES PREDEFINIDAS

Funciones predefinidas

- MATLAB/Octave/Scilab tienen un gran número de funciones incorporadas.
- Estas son *predefinidas* o *intrínsecas* (incorporadas en el propio código ejecutable del programa), por ende rápidas y eficientes.
- Su formato general es:

[valor(es) de retorno] = nombre función(argumento(s))

- Se llaman utilizando su nombre en una expresión o como comando.
- Además existen funciones definidas en archivos ***.m** (MATLAB/Octave) o ***.sci** (Scilab) que han sido aportadas por usuarios o bien circulan en la web (ver páginas del software).

Funciones predefinidas

Ejemplos:

```
>> max(y);
ans = 9           (máxima componente de y)

>> [ maxx, posmax ] = max(x)
maxx = 4          (máxima componente de x)
posmax = 1        (índice de la máxima componente de x)

>> a = cos(pi/2) - sin(pi/2)
a = -1.0000       (funciones trigonométricas)

>> v = [ 1 2 3], nv = sqrt(sum(v.^2))
v = [ 1 2 3 ]
nv = 3.7417       (norma euclídea del vector v)
```

Funciones predefinidas

[xm,im]=max(x): componente máxima xm de un vector e índice que ocupa im.

[xn,in]=min(x): componente mínima xn de un vector e índice que ocupa in.

n=length(x) si x vector, devuelve la cantidad de elementos.

sum(x): suma los elementos de un vector.

cumsum(x): vector suma acumulada de elementos de un vector.

mean(x): valor medio de los elementos de un vector.

prod(x): producto de los elementos de un vector.

cumprod(x): vector producto acumulado de elementos de vector.

[y,in]=sort(x): ordenación creciente de elementos de un vector x. Devuelve el vector ordenado y, junto a un vector in con posiciones iniciales en x de los elementos en el vector ordenado y.

Otras funciones: polinomios

- Un polinomio se puede definir por su vector de coeficientes:

```
>> pol = [1 0 -8 6 -10]
pol = 1 0 -8 6 -10  $\mapsto p(x) = x^4 - 8x^2 + 6x - 10$ 
```

- Para evaluarlo en un determinado valor de x usamos **polyval**:

```
>> polyval(pol,1)
ans = -11  $\mapsto p(1)$ 
```

- Para calcular sus raíces usamos **roots**:

```
>> roots(pol)
ans =
-3.2800  $\mapsto$  raíces reales
2.6748
0.3026 + 1.0238i  $\mapsto$  raíces complejas
0.3026 - 1.0238i
```

Algunas funciones del *Análisis Matemático*

$\sin(x)$: seno.

$\cos(x)$: coseno.

$\tan(x)$: tangente.

$\text{asin}(x)$: arco seno.

$\text{acos}(x)$: arco coseno.

$\text{atan}(x)$: arco tangente.

$\sinh(x)$: seno hiperbólico.

$\cosh(x)$: coseno hiperbólico.

$\tanh(x)$: tangente hiperbólica.

$\text{asinh}(x)$: arco seno hiperbólico.

$\text{acosh}(x)$: arco coseno hiperbólico.

$\text{atanh}(x)$: arco tangente hiperbólica.

Más funciones del *Análisis Matemático*

log(x): logaritmo natural.

log10(x): logaritmo decimal.

exp(x): función exponencial.

abs(x) valor absoluto.

sqrt(x): raíz cuadrada.

sign(x): devuelve -1 si $x < 0$, 0 si $x = 0$ y 1 si $x > 0$.

rem(x,y): resto de la división entre x e y .

round(x): blueondeo hacia el entero más próximo.

fix(x): blueondea hacia el entero más próximo a 0.

floor(x): valor entero más próximo hacia $-\infty$.

ceil(x): valor entero más próximo hacia $+\infty$.

real(z): parte real de $z = Re(z) + ilm(z)$.

imag(z): parte imaginaria de $z = Re(z) + ilm(z)$.

Algunas funciones del *Algebra Lineal*

[m,n] = size(A) número de filas m y columnas n de una matriz A .

norm(x,1): norma 1 de un vector o matriz (veremos $\|x\|_1$ y $\|A\|_1$).

norm(x): norma 2 de un vector o matriz (veremos $\|x\|_2$ y $\|A\|_2$).

norm(x,inf): norma infinito de un vector o matriz (veremos $\|x\|_\infty$ y $\|A\|_\infty$).

det(A): determinante de una matriz $\det(A)$.

inv(A): inversa de una matriz A^{-1} .

eig(A): autovalores de una matriz $\lambda_1, \dots, \lambda_n$.

[V,D]=eig(A): autovectores y autovalores de una matriz.

cond(A): número de condición de una matriz (veremos $\kappa(A)$).

[L,U]=lu(A): factorización LU de una matriz $A = L \cdot U$ (veremos).

Más funciones del *Algebra Lineal*

zeros(m,n): forma matriz de ceros de tamaño $m \times n$.

ones(m,n): forma matriz de unos de tamaño $m \times n$.

diag(x): matriz diagonal con elementos diagonales del vector **x**.

diag(A): forma vector con los elementos diagonales de la matriz **A**.

rand(n): forma matriz de tamaño $n \times n$ y de números aleatorios en $[0, 1]$ (con distribución uniforme).

rand(m,n): idem pero tamaño $m \times n$.

hilb(n): matriz de Hilbert de orden $n \times n$ (en MATLAB/Octave).

Notemos que algunas funciones se aplican indistintamente a escalares **x**, vectores **x** o matrices **A**.

¿Qué se entiende por “vectorización”?

El acto de transformar un código desarrollado para data escalar para usarlo en data no escalar, se llama **vectorización**.

VENTAJAS:

- Impacta enormemente en la velocidad de cálculos (10 o incluso 1000 veces más rápido).
- Permite escribir en forma sintética y general códigos.
- MATLAB/Octave/Scilab internamente tienen automatizado paralelizar operaciones lógicas y aritméticas sobre datos (*paralelización automática implícita*).
- La mayoría de las funciones predefinidas están paralelizadas.

Ejemplos de vectorización

- Operamos sobre los elementos de un vector o matriz usando funciones predefinidas:

```
>> sqrt([1,4; 9,16])
```

```
ans =
```

```
1 2
```

```
3 4      (raíz cuadrada componente a componente)
```

```
>> abs([0,1,2,-5,-6,-7])
```

```
ans =
```

```
0 1 2 5 6 7      (valor absoluto componente a componente)
```

*¡Si quisiéramos hacer esto en otros lenguajes
habría que programar estructuras de bucles!*

PROGRAMACIÓN

¿Cómo programar en MATLAB/Octave/Scilab?

- Podemos programar muy fácilmente con estos softwares aunque no tienen tantas posibilidades como otros lenguajes más avanzados (Fortran, C, C++, Python, Julia).
- Un programa se escribe en un archivo ***.m** en MATLAB/Octave o ***.sce, *.sci** en Scilab.
- Estos archivos se crean y se modifican con cualquier Editor de textos pero es recomendable usar el editor propio.
- Existen dos tipos de archivos:

funciones
archivos de comandos (scripts)

Estos últimos también se conocen como **guiones** o **libretos**.

¿Cómo programar en MATLAB/Octave/Scilab?

- Disponemos de sentencias para:

Bifurcaciones: realizan una o más operaciones según se cumpla o no determinada condición (**if**, **switch/select**).

Bucles: permiten repetir o iterar las mismas operaciones sobre datos distintos (**for**, **while**).

- El carácter porcentual (%) indica comienzo de un comentario en una línea de comandos (todo lo que va desde ese carácter hasta el fin de la línea es comentario).

¡Es importante comentar nuestros programas!

Programación en MATLAB/Octave/Scilab: *if*

- En su forma más simple, la sentencia de bifurcación **if** es:

```
if [condición]
    sentencias
end
```

- Existe también la bifurcación múltiple:

```
if [condición 1]
    bloque 1
elseif [condicion 2]
    bloque 2
else
    bloque 3
end
```

Programación en MATLAB/Octave/Scilab: *switch* o *select*

- La sentencia de bifurcación **switch** realiza una función análoga a un conjunto de **if ... elseif** concatenando casos:

MATLAB/Octave:

```
switch [variable]
case valor 1
    bloque 1
case {valor 2, ..., valor n}
    bloque 2
otherwise
    bloque 3
end
```

Scilab:

```
select [variable]
case valor 1
    bloque 1
case {valor 2, ..., valor n}
    bloque 2
otherwise
    bloque 3
end
```

Primero se expresa la *[variable]* escalar o string. Luego se compara con *valor 1* y si es igual se ejecuta el *bloque 1* de sentencias (es posible agrupar varios casos entre llaves). Si ninguno es igual se ejecutan las sentencias del caso *otherwise*.

Programación en MATLAB/Octave/Scilab:

- La sentencia de bucle **for** repite un conjunto de sentencias un número determinado de veces:

```
for i=inicio:paso:fin  
    sentencias  
end
```

- Otra forma:

```
for i=n:-1:1  
    sentencias  
end
```

- Otra:

```
for x=[ 10  5.5  1  -3.5  -8 ]  
    sentencias  
end
```

Programación en MATLAB/Octave/Scilab:

- La estructura de bucle **while** es similar a otros lenguajes de programación:

```
while [condicion]
    sentencias
end
```

- Puede hacerse:

```
while 1
    sentencias
    condición de salida
    break
end
```

- break**: termina la ejecución del bucle interno.
- return**: permite salir de una función antes de llegar a la última orden.

¿Cómo almacenar variables en archivos?

- En el trabajo con *Métodos Numéricos* aplicados a problemas reales o a problemas científicos, muchas veces encontramos que nuestra data (“input”) viene en formato: ***.dat**, ***.txt** u otro tipo.
- Es importante poder leer dichos datos, almacenarlos en alguna variable y luego poder trabajar con estos valores almacenados.
- También es importante que nuestra salida (“output”) se guarde en algún espacio de trabajo o archivo para que luego puede ser leído nuevamente para tabular, graficar o analizar errores.
- Esta tarea no resulta sencilla cuando los archivos son grandes.

*Recordemos que cuando cerramos la sesión,
el contenido de las variables se borra automáticamente.*

¿Cómo leer datos desde archivos?

- Para recuperar los datos almacenados en un archivo podemos utilizar el comando **load** (MATLAB/Octave) o **read** (Scilab) que lee datos del archivo y lo almacena en una variable:

MATLAB, Octave:

```
>> load datos.dat
```

Scilab:

```
>> read('datos.dat',m,n)
```

Guardamos el contenido del archivo `datos.dat` existente en el directorio, en una variable llamada **datos** (en Scilab debe especificarse su dimensión).

- Si deseamos que la variable tenga distinto nombre:

```
>> A = load('matriz.dat')
```

```
>> A = read('matriz.dat',m,n)
```

Guardamos el contenido del archivo `matriz.dat` en la variable **A**.

- En MATLAB si tipeamos:

```
>> load
```

Recuperamos las variables definidas en la sesión anterior guardadas previamente con el comando `save`.

¿Cómo guardar datos en archivos?

- Para grabar datos en un archivo se utiliza el comando **save**:

```
>> save
```

Creamos un archivo binario con el nombre y valor de todas las variables definidas en la sesión.

```
>> save datos A, x, y
```

Guardamos las variables A, x e y en el archivo binario dato.

```
>> save datos.dat A -ascii
```

Guardamos la matriz A en el archivo datos.dat en formato ascii

- Cada elemento de A se almacena con 8 cifras decimales.

En cambio, si deseamos almacenar 16 cifras decimales:

```
>> save datos.dat A -ascii -double
```

En formato ASCII sólo se guardan valores y no información tal como nombres de matrices y/o vectores.

Bibliografía

- **[MaFi2000]** J.M. Mathews & K.D. Fink, *Métodos Numéricos con MATLAB*, 3era Ed., Pearson Prentice Hall, Madrid, 2000.
- **[Mole2015]** C.B. Moler, *Numerical computing with MATLAB*, SIAM, Philadelphia, 2015.
- **[Ponz2024]** L. Ponzellini Marinelli, *Taller de Álgebra Lineal en Scilab*, FCEIA, UNR, 2024.
- **[SPMK2023]** J.A. Semitiel, L. Ponzellini Marinelli & A.I. Kurdobrin, *Notas de Clases de Matemática Aplicada de Ingeniería Eléctrica e Ingeniería Electrónica*, FCEIA, UNR, 2023.
- **[SiSo2010]** J.W. Signorelli, J. Sorribas, *Notas de Clases de Informática Aplicada y Métodos Computacionales*, FCEIA, UNR, 2010.